

Observabilidad & KQL — DocumentIA

Objetivo: Un operador o dev puede diagnosticar issues usando KQL. Todas las queries están listas para copiar y pegar.

1. Introducción & Overview

Stack de Observabilidad

- **Azure Application Insights** (srbappiprodocai)
- **Log Analytics** (workspace adjunto a Applnsights)
- **Azure Monitor** (alertas, métricas, health checks)
- **Distributed Tracing** (correlation IDs via `Activity.Current?.TraceId`)

Arquitectura de Telemetría



Métricas Clave Monitoreadas

Métrica	Fuente	Propósito
CU.PrepareMs	AzureContentUnderstandingProvider	Tiempo preparación antes de CU
CU.LimiterWaitMs	AzureContentUnderstandingProvider	Cola local / backpressure
CU.AnalysisMs	AzureContentUnderstandingProvider	Tiempo real de Azure CU
CU.ParseMs	AzureContentUnderstandingProvider	Parseo respuesta
CU.Attempts	AzureContentUnderstandingProvider	Reintentos
Confianza	GptClasificarDataProvider	Score clasificación GPT (0.0–1.0)
ConfianzaDI	DocumentIntelligenceProvider	Score Document Intelligence (0.0–1.0)
Duration by Activity	Durable Functions	Latencia por step
Execution State	DocumentProcessOrchestrator	Estado orquestación

Eventos Custom

- `CU.TransientError` – Reintento de transient error
- `CU.HardTimeout` – Timeout permanente en CU
- `CU.CircuitOpen` – Circuit breaker activado
- `CU.CircuitClosed` – Circuit breaker normalizado
- `CU.CircuitFailover` – Switchover a fallback
- `CU.CircuitRejected` – Request rechazado por circuit
- `GDC.UploadSuccess`, `GDC.UploadFailed` – Subida a GDC
- `Classification.Phase1Success`, `Classification.Phase1Failed` – Fases GPT
- Otros eventos: `Orchestration.Started`, `Orchestration.Failed`, `Duplicado.Detected`

2. Application Insights Dashboards

2.1 Workbooks Predefinidos

CU Performance Workbook (Disponible)

Ubicación: `docs/observabilidad/workbooks/documentia-cu-performance.workbook.json`

Contenido:

- Timeline de CU latencias (PrepareMs, LimiterWaitMs, AnalysisMs, ParseMs)
- Distribución de reintentos
- Circuit breaker events
- Correlación entre load y backpressure

Cómo importar:

1. Abrir Azure Portal → Application Insights `srbappiprodocai`
2. **Workbooks** → + **New**
3. **Advanced Editor** (arriba a la derecha)
4. **Paste** el JSON desde `docs/observabilidad/workbooks/documentia-cu-performance.workbook.json`
5. **Done**
6. **Guardar como** `DocumentIA - CU Performance`

Recomendaciones para Otros Workbooks

- **Clasificación Metrics** – Confianza por tipología, distribution de confianza
- **Orquestación Timeline** – Duration por activity, estado final, retries
- **Cost Breakdown** – CU tokens, GPT tokens, requests by provider
- **Error Heatmap** – Errores por activity, timeline de fallas

2.2 Cómo Acceder & Usar

URL Directa a Application Insights:

```
https://portal.azure.com/#@sareb.com/resource/subscriptions/{subscription-id}/resourcegroups/SRBRGDOCSAIPROD/providers/microsoft.insights/components/srbappiprodocai/overview
```

Seleccionar Time Range:

- Arriba a la derecha: **Last 24 hours**, **Last 7 days**, **Last 30 days**, etc.
- Custom: click en el rango de fecha para abrir picker

Exportar Resultados:

- Queries → **Export** → CSV, Excel, JSON
- Workbooks → **Print** o **Download as PDF**

Compartir Dashboards:

- Workbook abierto → **Share** (arriba a la derecha)
- Copiar link o enviar a equipo

3. KQL Queries Fundamentales

Nota: Sustituir `ago(24h)` por el rango temporal deseado en todas las queries.

3.1 Disponibilidad & Health

Query 1: Uptime de API por Endpoint

```
// Uptime de API endpoints en últimas 24h
// Retorna: endpoint, total requests, failed (5xx), error rate %
requests
| where timestamp > ago(24h)
| where name startswith "POST" or name startswith "GET"
| summarize TotalReqs=dcount(id), Failed=countif(resultCode >= "500"), SuccessReqs=countif(resultCode < "500")
  by tostring(name), tostring(resultCode)
| extend ErrorRate = (Failed * 100.0) / TotalReqs
| where ErrorRate > 0
| sort by ErrorRate desc
```

Cómo leer:

- Si `ErrorRate` > 5% → alertar
- Si `ErrorRate` > 10% → crítico
- Agrupar por endpoint para identificar qué API falla

Query 2: Health Check — Activities by Status

```
// Estado de actividades en orquestación
// Retorna: activity name, total, completed, failed, pendiente
customMetrics
| where name in ("Activity.Duration", "Activity.Status")
| extend ActivityName = tostring(customDimensions["ActivityName"]),
    ActivityStatus = tostring(customDimensions["Status"])
| where timestamp > ago(24h)
| summarize Total=dcount(id), Completed=countif(ActivityStatus == "Completed"),
    Failed=countif(ActivityStatus == "Failed"), Pending=countif(ActivityStatus == "Pending")
    by ActivityName
| sort by Failed desc
```

Cómo leer:

- Si `Failed` > 0 → investigar activity específica
- Si `Pending` > threshold → posible deadlock

Query 3: Service Health — CU Service Availability

```
// Disponibilidad del servicio Azure Content Understanding (últimas 24h)
// Retorna: event type, count, distribution
customEvents
| where timestamp > ago(24h)
| where name in ("CU.TransientError", "CU.HardTimeout", "CU.CircuitOpen", "CU.CircuitRejected")
| summarize Count=count() by name
| extend Severity = case(
    name == "CU.HardTimeout", "CRITICAL",
    name == "CU.CircuitOpen", "WARNING",
    name == "CU.CircuitRejected", "WARNING",
    name == "CU.TransientError", "INFO",
    "UNKNOWN"
)
| sort by Count desc
```

Cómo leer:

- `CU.HardTimeout` > 10 → contactar Azure CU support
- `CU.CircuitOpen` → sistema saturado, escalar replicas o reducir load
- `CU.CircuitRejected` → circuit abierto, esperar recovery

3.2 Performance

Query 4: P50, P95, P99 Latencies by Endpoint

```
// Percentiles de latencia (ms) por endpoint
// Retorna: endpoint, p50, p95, p99, avg
requests
| where timestamp > ago(24h)
| where resultCode == "200" // solo successful
| summarize
    P50=percentile(duration, 50),
    P95=percentile(duration, 95),
    P99=percentile(duration, 99),
    Avg=avg(duration),
    Count=count()
    by tostring(name)
| sort by P99 desc
```

Cómo leer:

- `P99` > 30000 ms (30s) → problema de latencia, investigar

- P50 vs P95 spread > 2x → inconsistencia, buscar outliers
- Si P99 sube súbitamente → caída de recursos (CPU/memoria)

Query 5: CU Performance — Breakdown Temporal

```
// Desglose de tiempos CU: prepare, limiter wait, analysis, parse
// Retorna: avg (ms) for each phase
customMetrics
| where name in ("CU.PrepareMs", "CU.LimiterWaitMs", "CU.AnalysisMs", "CU.ParseMs")
| where timestamp > ago(24h)
| summarize AvgMs=avg(value), MaxMs=max(value), MinMs=min(value), Count=count() by name
| sort by AvgMs desc
```

Cómo leer:

- Si CU.LimiterWaitMs > CU.AnalysisMs → **backpressure local**, aumentar concurrencia o replicas
- Si CU.AnalysisMs >> (PrepareMs + LimiterWaitMs + ParseMs) → **Azure CU lento**, contactar soporte
- CU.ParseMs alto → problema en mapeo respuesta, revisar código

Query 6: Throughput (Requests/min) — Trend

```
// Throughput por minuto en últimas 24h
// Retorna: bin temporal, requests count
requests
| where timestamp > ago(24h)
| summarize ReqCount=count() by bin(timestamp, 1m)
| extend ReqPerMin = ReqCount
| order by timestamp desc
```

Cómo leer:

- Gráfico en UI mostrará trend
- Buscar caídas abruptas (posible outage o error)
- Comparar con histórico (baseline) para anomalías

Query 7: Activity Duration Distribution

```
// Duration de cada activity en orquestación (últimas 24h)
// Retorna: activity name, avg duration (s)
customMetrics
| where name == "Activity.Duration"
| where timestamp > ago(24h)
| extend ActivityName = toString(customDimensions["ActivityName"])
| summarize AvgDurationS=avg(value/1000), MaxDurationS=max(value/1000), Count=count() by ActivityName
| sort by AvgDurationS desc
```

Cómo leer:

- Activities con AvgDuration > esperado → bottleneck
- Si MaxDuration >> AvgDuration → outliers, investigar causa
- Típicos:
 - ExtraerActivity (CU): 5–15s
 - ClasificarActivity (GPT): 2–8s
 - SubirGDCActivity: 1–5s

3.3 Errores & Exceptions

Query 8: Error Rate Trend (último 24h)

```
// Error rate por hora en últimas 24h
// Retorna: bin temporal, total requests, error count, error rate %
requests
| where timestamp > ago(24h)
| summarize TotalReqs=count(), ErrorCount=countif(resultCode >= "500")
  by bin(timestamp, 1h)
| extend ErrorRate = (ErrorCount * 100.0) / TotalReqs
| sort by timestamp desc
```

Cómo leer:

- Si ErrorRate sube > 5% → alerta
- Picos de error indican incidentes recientes
- Correlacionar con logs para identificar causa

Query 9: Top Error Messages (últimas 24h)

```
// Errores más frecuentes
// Retorna: error message, count, first occurrence
exceptions
| where timestamp > ago(24h)
| summarize Count=count(), FirstOccurrence=min(timestamp), LastOccurrence=max(timestamp) by toString(outerMessage)
| sort by Count desc
| limit 20
```

Cómo leer:

- Errores repetidos → problema sistemático
- New errors → investigar cambios recientes
- Stack trace disponible en detail view

Query 10: Errors by Activity

```
// Distribuir errores por activity
// Retorna: activity, error count, error message sample
customEvents
| where name startswith "Activity.Error"
| where timestamp > ago(24h)
| extend ActivityName = toString(customDimensions["ActivityName"]),
  ErrorMsg = toString(customDimensions["ErrorMessage"])
| summarize ErrorCount=count(), SampleError=any(ErrorMsg) by ActivityName
| sort by ErrorCount desc
```

Cómo leer:

- `ExtraerActivity` errors altos → CU issues, revisar circuit breaker
- `ClasificarActivity` errors → GPT/DI issue, revisar rate limits
- `SubirGDCActivity` errors → GDC connectivity, revisar Auth/network

Query 11: Circuit Breaker Status

```
// Estado del circuit breaker de CU (últimas 24h)
// Retorna: event timeline, circuit state changes
customEvents
| where timestamp > ago(24h)
| where name in ("CU.CircuitOpen", "CU.CircuitClosed", "CU.CircuitFailover")
| extend CircuitState = case(
    name == "CU.CircuitOpen", "OPEN",
    name == "CU.CircuitClosed", "CLOSED",
    name == "CU.CircuitFailover", "FAILOVER",
    "UNKNOWN"
)
| summarize Count=count() by CircuitState
| extend Status = case(
    CircuitState == "OPEN", "ALERT",
    CircuitState == "CLOSED", "OK",
    "?"
)
)
```

Cómo leer:

- Si OPEN count alto → desconexión frecuente
- Si FAILOVER eventos → failover a backup CU endpoint
- Correlacionar con `CU.CircuitRejected` para ver impact en requests

3.4 Clasificación — Métricas de Negocio

Query 12: Confianza Distribution (Todas las clasificaciones últimas 24h)

```
// Distribución de scores de confianza
// Retorna: confianza, count (histograma)
customMetrics
| where name == "Classification.Confidence"
| where timestamp > ago(24h)
| extend ConfidenceBucket = floor(value * 10) / 10 // bucket 0.0-0.1, 0.1-0.2, etc
| summarize Count=count() by ConfidenceBucket
| sort by ConfidenceBucket asc
```

Cómo leer:

- Si alta concentración en 0.0-0.3 → clasificador débil, revisar prompt
- Si 0.8-1.0 → confianza alta, calidad OK
- Distribución bimodal → dos comportamientos diferentes, investigar

Query 13: Confianza por Tipología (últimas 24h)

```
// Confianza promedio por tipología detectada
// Retorna: tipología, avg confianza, count
customMetrics
| where name == "Classification.Confidence"
| where timestamp > ago(24h)
| extend Tipologia = tostring(customDimensions["Tipologia"])
| summarize AvgConfianza=avg(value), Count=count(), MinConfianza=min(value), MaxConfianza=max(value) by Tipologia
| sort by AvgConfianza asc
```

Cómo leer:

- Tipologías con AvgConfianza < 0.6 → problema clasificación, revisar training
- Tipologías raras (Count < 5) → no hay suficientes ejemplos, aumentar corpus
- MaxConfianza = 1.0, MinConfianza = 0.0 → gran variabilidad, revisar feature engineering

Query 14: Clasificación Fallida — Baja Confianza vs Desconocido

```
// Desglose de clasificaciones que fallaron
// Retorna: tipo de fallo, count
customEvents
| where timestamp > ago(24h)
| where name in ("Classification.LowConfidence", "Classification.Unknown", "Classification.Error")
| summarize Count=count() by name
| extend Severity = case(
    name == "Classification.Error", "CRITICAL",
    name == "Classification.Unknown", "WARNING",
    name == "Classification.LowConfidence", "INFO",
    "?"
)
```

Cómo leer:

- **Unknown** alto → tipologías fuera del catálogo, ampliar modelo
- **LowConfidence** alto → aumentar umbral o mejorar prompt
- **Error** → contactar tech support

Query 15: Provider Comparison (GPT vs Document Intelligence)

```
// Comparar confianza entre GPT y Document Intelligence
// Retorna: provider, avg confianza, success rate
customMetrics
| where timestamp > ago(24h)
| where name in ("Classification.ConfidenceGPT", "Classification.ConfidenceDI")
| extend Provider = case(
    name == "Classification.ConfidenceGPT", "GPT",
    name == "Classification.ConfidenceDI", "Document Intelligence",
    "Unknown"
)
| summarize AvgConfidence=avg(value), Count=count(), P95=percentile(value, 95) by Provider
```

Cómo leer:

- Si GPT > DI en confianza → GPT más confiable para este dataset
- Si DI > GPT → Document Intelligence better fit
- Usar para routing: usar provider con mayor confianza

3.5 Orquestación (Durable Functions)

Query 16: Orchestration Execution Timeline (últimas 24h)

```
// Timeline de ejecuciones orquestadas
// Retorna: execution id, start, end, duration, final state
customEvents
| where name in ("Orchestration.Started", "Orchestration.Completed", "Orchestration.Failed")
| where timestamp > ago(24h)
| extend
    ExecutionId = tostring(customDimensions["ExecutionId"]),
    State = name
| summarize
    StartTime=min(timestamp),
    EndTime=max(timestamp),
    States=make_set(State)
    by ExecutionId
| extend Duration = (EndTime - StartTime) / 1s
| sort by StartTime desc
| limit 50
```


Cómo leer:

- Duration > 5 min → revisar qué activity es bottleneck
 - Failed estado → revisar exception trace
 - Pending (sin Completed) → posible timeout o deadlock
-

Query 17: Activity Success Rate (últimas 24h)

```
// Success rate por activity en orquestación
// Retorna: activity name, total, success, failed, success rate %
customEvents
| where name startswith "Activity."
| where timestamp > ago(24h)
| extend ActivityName = tostring(customDimensions["ActivityName"]),
    ActivityStatus = tostring(customDimensions["Status"])
| summarize
    Total=count(),
    Success=countif(ActivityStatus == "Completed"),
    Failed=countif(ActivityStatus == "Failed")
    by ActivityName
| extend SuccessRate = (Success * 100.0) / Total
| sort by SuccessRate asc
```

Cómo leer:

- SuccessRate < 95% → crítico
 - SuccessRate 95–99% → monitorear
 - SuccessRate > 99% → OK
-

Query 18: Retry Count Distribution

```
// Distribución de reintentos por activity
// Retorna: activity, retry count, frequency
customMetrics
| where name == "Activity.RetryCount"
| where timestamp > ago(24h)
| extend ActivityName = tostring(customDimensions["ActivityName"])
| summarize
    AvgRetries=avg(value),
    MaxRetries=max(value),
    Executions=count()
    by ActivityName
| sort by MaxRetries desc
```

Cómo leer:

- AvgRetries > 2 → demasiados reintentos, revisar timeout o transient failures
 - MaxRetries = circuit limit → circuit breaker activándose, aumentar límite o revisar health
-

3.6 Recursos & Costs

Query 19: Function App CPU & Memory Usage

```
// Uso de recursos (CPU, memoria) en últimas 24h
// Retorna: timestamp bin, avg cpu %, avg memory %
performanceCounters
| where timestamp > ago(24h)
| where counterName in ("% Processor Time", "% Available Memory")
| summarize
    AvgValue=avg(value),
    MaxValue=max(value)
    by bin(timestamp, 15m), counterName
| sort by timestamp desc
```

Cómo leer:

- CPU > 80% → aumentar replicas o optimizar código
- Memory > 90% → posible leak, revisar telemetry client disposal
- Peaks correlacionan con high throughput → escalar automáticamente

Query 20: Cost Estimation — Tokens by Provider

```
// Estimación de costo de tokens (GPT, DI)
// Retorna: provider, token count estimate
customMetrics
| where name in ("GPT.InputTokens", "GPT.OutputTokens", "DI.Tokens")
| where timestamp > ago(24h)
| extend Provider = case(
    name startswith "GPT", "Azure OpenAI",
    name startswith "DI", "Document Intelligence",
    "Unknown"
)
| summarize TotalTokens=sum(value), CallCount=count() by Provider
| extend EstimatedCost = case(
    Provider == "Azure OpenAI", (TotalTokens / 1000.0) * 0.015, // GPT-4o Mini: $0.015/1k tokens
    Provider == "Document Intelligence", (TotalTokens / 1000.0) * 0.05, // DI: $0.05/1k tokens
    0
)
)
```

Cómo leer:

- Cost trend arriba → revisar si hay loops infinitos o calls innecesarios
- Correlacionar con throughput: si costo sube pero throughput igual → ineficiencia

4. Queries Avanzadas

Query 21: End-to-End Trace Reconstruction

```
// Reconstruir un request completo: entrada → activities → salida
// Requisito: conocer el OperationId o ExecutionId
let operationId = "YOUR_OPERATION_ID_HERE"; // Reemplazar
traces
| union (requests | extend customDimensions["Phase"] = "HTTP")
| where operation_Id == operationId or toString(customDimensions["ExecutionId"]) == operationId
| project
    timestamp,
    message,
    severityLevel,
    operation_Id,
    customDimensions["ActivityName"],
    customDimensions["Phase"]
| sort by timestamp asc
```

Cómo usar:

- 1. Obtener `OperationId` del error del cliente o log inicial
- 2. Reemplazar `YOUR_OPERATION_ID_HERE`
- 3. Ejecutar query
- 4. Leer timeline de eventos

Query 22: Anomaly Detection — Latency Spike

```
// Detectar anomalías en latencia (últimas 24h)
// Retorna: timestamp, avg latency, anomaly score
requests
| where timestamp > ago(24h)
| where resultCode == "200"
| summarize AvgLatencyMs=avg(duration) by bin(timestamp, 5m)
| extend
    Baseline=5000, // Reemplazar con tu baseline (ms)
    Deviation = (AvgLatencyMs - Baseline) / Baseline * 100,
    AnomalyScore = iif(Deviation > 50, "HIGH", iif(Deviation > 20, "MEDIUM", "OK"))
| where AnomalyScore != "OK"
| sort by timestamp desc
```

Cómo usar:

- 1. Reemplazar `Baseline` con tu latencia esperada normal
- 2. Ejecutar query
- 3. Identificar cuándo anomalías empezaron

5. Alertas & Notificaciones

5.1 Alertas Recomendadas

Alert	Condition	Threshold	Severity	Action
HTTP 5xx Error Rate	<code>exceptions count / requests count > threshold</code>	> 5% in 5m	CRITICAL	Teams + PagerDuty
CU Timeout	<code>CU.HardTimeout events</code>	> 10 in 30m	HIGH	Teams
Circuit Breaker Open	<code>CU.CircuitOpen events</code>	> 1 in 5m	HIGH	Teams + Escalate
Classification Low Conf	<code>Classification.LowConfidence</code>	> 50% in 1h	MEDIUM	Teams (daily digest)
Orchestration Timeout	Activities duration > 30m	> 0	MEDIUM	Teams + Logs
Memory Usage High	<code>% Available Memory</code>	< 10%	MEDIUM	Auto-scale + Teams
Provider Failure	GPT or DI provider unavailable	> 1 incident in 5m	CRITICAL	Failover + Teams
Cost Spike	Token count anomaly	+50% vs baseline	LOW	Teams (weekly)

5.2 Crear una Alerta Custom

Paso 1: Abrir Application Insights

```
https://portal.azure.com → Application Insights → srbappiprodocai → Alerts
```

Paso 2: Crear Nueva Regla

- 1. + New Alert Rule
- 2. **Condition:** Seleccionar métrica o query personalizada

3. **Threshold:** Definir umbral (ej. error rate > 5%)
4. **Time Window:** 5 min (o la que necesites)
5. **Frequency:** Every 1 minute (o menor)

Paso 3: Configurar Acción

1. **Action Group:** Crear o usar existente
2. **Notification Type:**
 - Email
 - SMS
 - Teams Webhook
 - Azure Function (para acciones automáticas)
3. **Test Notification:** Verificar que llega

Paso 4: Guardar

- **Alert Rule Name:** Alert: HTTP 5xx > 5%
- **Description:** Critical: Production API errors increasing
- **Save**

6. Debugging Avanzado

6.1 Distributed Tracing — Seguir un Request

Caso: Usuario reporta "Document got stuck after 5 minutes"

Paso 1: Obtener el ID del documento

Consultar DB o logs del cliente: documento name, ID contrato, etc.

Paso 2: Buscar en Applnsights

```
// Query: Encontrar todas las ejecuciones de un documento
requests
| where timestamp > ago(48h)
| where tostring(customDimensions["DocumentName"]) == "tu_documento.pdf"
| project
    timestamp,
    operation_Id,
    resultCode,
    duration,
    customDimensions["ExecutionState"]
```

Paso 3: Obtener OperationId

```
// Query: Get operation ID
let documentName = "tu_documento.pdf";
requests
| where tostring(customDimensions["DocumentName"]) == documentName
| project operation_Id
| take 1
```

Paso 4: Reconstruir timeline completa

```
// Query: Timeline completa del documento (desde HTTP request → activities)
let opId = "YOUR_OPERATION_ID";
traces
| union (requests | extend customDimensions["ActivityName"] = name)
| union exceptions
| where operation_Id == opId
| project
    timestamp,
    message,
    severityLevel,
    customDimensions["ActivityName"],
    customDimensions["Status"],
    toString(outerException)
| sort by timestamp asc
```

Paso 5: Identificar bottleneck

```
// Query: Activity durations
let opId = "YOUR_OPERATION_ID";
customEvents
| where operation_Id == opId
| where name startswith "Activity."
| extend ActivityName = customDimensions["ActivityName"],
    StartTime = todatetime(customDimensions["StartTime"]),
    EndTime = todatetime(customDimensions["EndTime"])
| project ActivityName, StartTime, EndTime, Duration = (EndTime - StartTime)
| sort by StartTime asc
```

Output Esperado:

ActivityName	StartTime	EndTime	Duration
ExtraerActivity	2026-06-10T10:00:00Z	2026-06-10T10:05:30Z	330s (LONG)
NormalizarActivity	2026-06-10T10:05:30Z	2026-06-10T10:05:35Z	5s
ClasificarActivity	2026-06-10T10:05:35Z	2026-06-10T10:05:42Z	7s

Conclusión: ExtraerActivity tardó 5.5 min. CU está lento. Revisar CU.AnalysisMs .

6.2 Debugging Caso Real: "Classification Confidence = 0"

Problema: Documento clasificado con confianza 0.

Paso 1: Buscar el evento

```
// Query: Clasificaciones con confianza 0
customMetrics
| where name == "Classification.Confidence"
| where value == 0
| where timestamp > ago(24h)
| project
    timestamp,
    operation_Id,
    customDimensions["DocumentName"],
    customDimensions["Tipologia"],
    customDimensions["ErrorMsg"]
```

Paso 2: Analizar logs de GptClasificarDataProvider

```
// Query: Ver logs de GPT clasificación
traces
| where message contains "Clasificación GPT" or message contains "GPT Phase"
| where operation_Id == "YOUR_OPERATION_ID"
| project timestamp, message, severityLevel
| sort by timestamp asc
```

Paso 3: Revisar respuesta GPT

```
// Query: Respuesta raw de GPT
traces
| where message contains "phase1ResponseText" or message contains "GPT Response"
| where operation_Id == "YOUR_OPERATION_ID"
| project message
```

Posibles Causas:

- **JSON inválido** → GPT devolvió formato incorrecto
- **Timeout** → GPT tardó > limit
- **Low confidence self-reported** → GPT mismo reportó confianza baja
- **Missing catalog** → Tipología no en catálogo

Acciones:

1. Si timeout → aumentar timeout en orchestrator
2. Si JSON inválido → revisar prompt, ejecutar test manual
3. Si confianza baja → revisar contexto del documento (texto corrupto?)

6.3 Debugging Caso Real: "Circuit Breaker Open for 1 hour"

Problema: CU circuit breaker abierto, documentos encolados.

Paso 1: Identificar cuándo se abrió

```
// Query: Cuando se abrió el circuit breaker
customEvents
| where name == "CU.CircuitOpen"
| where timestamp > ago(48h)
| project timestamp, customDimensions["Reason"], customDimensions["FailureCount"]
| sort by timestamp desc
| limit 5
```

Paso 2: Ver intentos fallidos previos

```
// Query: CU errors antes de circuit open
customEvents
| where name in ("CU.TransientError", "CU.HardTimeout")
| where timestamp > ago(2h)
| summarize Count=count() by bin(timestamp, 5m), name
| sort by timestamp asc
```

Paso 3: Revisar logs de CU

```
// Query: Logs de Azure Content Understanding
traces
| where message contains "AnalyzeBinaryAsync" or message contains "CU API"
| where timestamp > ago(2h)
| project timestamp, message, severityLevel
| sort by timestamp desc
| limit 20
```

Posibles Causas:

- **CU service down** → 5xx errors, contactar Azure
- **Rate limit hit** → 429 errors, implementar backoff
- **Network issue** → transient errors, esperar recovery
- **Bad document** → malformed PDF causing hangs

Acciones:

1. **Inmediato:** Incrementar circuit breaker timeout si CU se está normalizando
2. **Short-term:** Escalar issue a Azure CU support
3. **Long-term:** Mejorar circuit breaker configuration (thresholds, backoff strategy)

7. Troubleshooting Rápido

Síntoma	Queries a Ejecutar	Probable Causa	Acción
API devuelve 500	Query 8, Query 9	Code exception	Revisar stack trace
Documentos tardan > 5m	Query 6, Query 18	Activity lento	Ver duración por activity
Confianza = 0 en muchos docs	Query 12, ver logs GPT	GPT fallo o timeout	Revisar GPT provider
CU.CircuitOpen events	Query 11, Query 3	CU service unstable	Escalate Azure, aumentar timeout
Memory cresce continuamente	Query 19	Memory leak	Revisar TelemetryClient disposal
Cost dispara 2x	Query 20	Tokens se duplican	Revisar payload size, prompts

8. Mejores Prácticas

Para Devs

1. Usa correlation IDs

```
var traceId = Activity.Current?.TraceId.ToString();
_logger.LogInformation("Processing {DocumentName} with TraceId={TraceId}",
    documentName, traceId);
```

2. Track custom metrics

```
_telemetryClient.TrackMetric("MyCustomDuration", stopwatch.ElapsedMilliseconds);
```

3. Incluye contexto en exceptions

```
var properties = new Dictionary<string, string>
{
    { "DocumentName", documentName },
    { "ActivityName", "ClasificarActivity" }
};
_telemetryClient.TrackException(ex, properties);
```

Para Operadores

1. Revisar alertas diariamente

- Email digest o Teams notification
- Actuar sobre CRITICAL dentro de 1h

2. Ejecutar queries semanales

- Confianza trend
- Activity duration trend
- Cost estimation

3. Mantener dashboards actualizados

- Refrescar weekly para validar datos
- Añadir nuevas métricas según necesidad

En Caso de Incidente

1. **Primeros 5 min:** Identificar si es infrastructure o application
2. **Próximos 15 min:** Aislar scope (API vs Activity vs Provider)
3. **Próximos 30 min:** Root cause analysis usando KQL
4. **Post-incident:** Update runbook, post-mortem

Apéndice: Variables Temporales en KQL

Rango	Variable
Última 1 hora	<code>ago(1h)</code>
Últimas 24 horas	<code>ago(24h)</code>
Últimos 7 días	<code>ago(7d)</code>
Últimos 30 días	<code>ago(30d)</code>
Hoy	<code>startofday(now)</code>
Semana pasada	<code>startofweek(now - 7d)</code>

Apéndice: Estructura de CustomDimensions

Cuando instrumentes código, añade propiedades en `customDimensions` :

```
var properties = new Dictionary<string, string>
{
    { "DocumentName", document.Name },
    { "ActivityName", "ExtraerActivity" },
    { "Status", "Completed" },
    { "Provider", "AzureContentUnderstanding" },
    { "TipologiaDetectada", "ESCR-06" },
    { "ExecutionId", context.InstanceId } // Durable Functions
};
```

Luego, en KQL:

```
customMetrics
| extend ActivityName = tostring(customDimensions["ActivityName"])
| where ActivityName == "ExtraerActivity"
```

Apéndice: Recursos Externos

- **Azure Monitor & Application Insights Docs:** <https://docs.microsoft.com/en-us/azure/azure-monitor/>

- **KQL Reference:** <https://learn.microsoft.com/en-us/azure/data-explorer/kusto/query/>
 - **Durable Functions Diagnostics:** <https://docs.microsoft.com/en-us/azure/azure-functions/durable/durable-functions-diagnostics>
 - **Cost Analysis:** <https://portal.azure.com> → Cost Management + Billing
-

Última actualización: 2026-06-10